CHAPTER 2

METHODOLOGY

This chapter describes the research methodology employed in designing, implementing, and evaluating SynapseOS — a Linux distribution that replaces the conventional graphical userland with a conversational interface layer. The methodology is organized into three sections: Dataset, which defines the data to be collected and how it will be gathered; Procedure, which describes the step-by-step process of building the system and conducting the study; and System Testing, which defines the evaluation design, metrics, and statistical approach used to compare SynapseOS against the conventional Linux desktop.

CONCEPTUAL FRAMEWORK

SynapseOS operates as a conversational shell layer positioned between the user and the Linux command-line environment. The user issues natural language commands through a persistent chat interface. An intent parsing pipeline — powered by a locally-hosted small language model — translates each utterance into a shell command, which is executed in a bash subprocess. Output is captured and rendered back in the conversational interface. The user issues intent in natural language and receives results in natural language; the shell command is an implementation detail invisible to the user. This architecture is illustrated in Figure 2.1.

To illustrate: if a user says "find all PDF files in my Downloads folder and show their sizes," the intent parser generates the command `find ~/Downloads -name '*.pdf' -exec du -sh {} \;`. SynapseOS executes this in a bash subprocess, captures the output, and displays it in the conversational interface — the user never sees the command itself. The same flow applies to any operation expressible through the Linux command-line toolchain.

The framework is evaluated through a controlled user study in which participants complete a fixed set of tasks under two conditions: using SynapseOS as the interface, and using their own primary operating system — Windows, macOS, or Linux — as the baseline. This expert baseline design pits SynapseOS against the environment each participant already knows and uses daily, generating empirical data on task completion time, error rate, and user satisfaction across the real-world OS landscape [17].

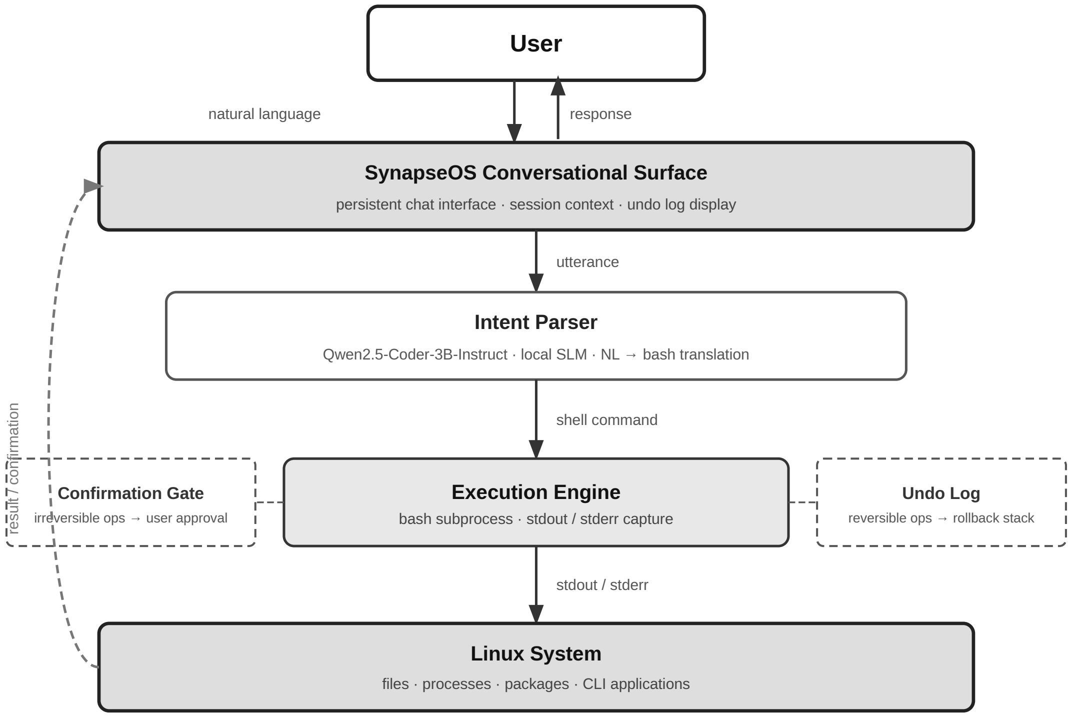

Figure 2.1. Conceptual framework of SynapseOS. The user issues a natural language command to the conversational surface, which routes the utterance to the intent parser (Qwen2.5-Coder-3B-Instruct, local SLM). The intent parser produces a shell command dispatched to the execution engine (bash subprocess). Irreversible operations are intercepted by the confirmation gate; reversible operations are logged to the undo stack. Command output (stdout/stderr) is captured, rendered in the conversational surface, and presented back to the user — completing the interaction loop.

SYSTEM ENVIRONMENT SPECIFICATIONS

The hardware and software environment on which SynapseOS is deployed and evaluated is specified in Table 2.1. All participants in the user study interact with the same physical machine configuration to control for hardware variability.

Table 2.1. System environment specifications

Component	Specification
Operating System	Debian 13 (Trixie)

Processor	x86-64 with AVX2 support (Intel 4th gen / AMD Ryzen 1st gen or newer); 2 vCPU minimum
RAM	4 GB minimum; 8 GB recommended
Storage	10 GB available (OS + model + application)
Display	Any monitor or remote terminal; no display server required (TUI interface)
Language Model Backend	Qwen2.5-Coder-3B-Instruct, Q4_K_M quantization (~1.8 GB); no GPU required; pluggable cloud LLM backend (optional, user-configured)
Inference Server	Ollama (model lifecycle management, REST API at localhost:11434)
Runtime Language	Go (session manager, TUI, bash execution, Ollama API client)
Build / Evaluation Language	Python 3.12 (LoRA fine-tuning, dataset preparation, statistical analysis)

The minimum specification of 2 vCPU and 4 GB RAM is intentional. Because SynapseOS executes only shell-expressible operations and its conversational surface is a text-based TUI, the system imposes no display server dependency and runs on commodity cloud server hardware. A deployment on a minimal virtual private server (VPS) — or accessed remotely over SSH — is within scope, distinguishing SynapseOS from graphical desktop alternatives that require substantially more capable hardware.

SECTION 1: DATASET

The data gathered for this study falls into three categories: benchmark task data used to evaluate the system in a standardized environment, user study data collected from human participants in a controlled setting, and system performance data captured automatically through built-in telemetry during all sessions.

1.1 What to Gather

a) Benchmark Task Suite

The primary benchmark dataset is the OSWorld task suite [17], which provides standardized real-computer tasks across shell operations, file management, web browsing, and application-level interactions. OSWorld is the only existing benchmark that evaluates computer-using agents in real operating system environments, making it the most appropriate external standard for SynapseOS evaluation. In addition to OSWorld tasks, a custom cross-platform task suite is developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing — each task defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine. Each task in both suites is documented with a description, expected outcome, task category, and difficulty level.

b) User Study Data

The following data types are collected from each participant during the user study:

- **Task completion time:** the elapsed time from when the task prompt is presented to when the participant signals task completion or the session logs a terminal event
- **Error rate:** the count of failed tasks, incorrect outcomes, and recovery attempts per participant per condition
- **User satisfaction scores:** post-study responses to the System Usability Scale (SUS) and the NASA Task Load Index (NASA-TLX), both validated instruments for measuring perceived usability and cognitive workload respectively
- **Qualitative feedback:** responses gathered through a semi-structured post-study interview covering perceived ease of use, trust in the system, and comparison between the two conditions

c) System Performance Data

The following telemetry data is captured automatically by SynapseOS throughout all sessions:

- Command execution latency: time elapsed from utterance submission to bash subprocess completion
- Undo log events: count and type of operations logged to the undo stack
- Confirmation prompts triggered: count of irreversible operations that required user confirmation

- Conversational turns per task: the number of utterances required to complete each task
- Intent parsing accuracy: whether the system correctly identified the user's intent on the first attempt

1.2 How to Gather

a) Benchmark Tasks

SynapseOS is deployed on the Debian 13 (Trixie) environment matching the OSWorld evaluation setup [17]. Each benchmark task is executed through the conversational interface, and results are captured programmatically through the built-in telemetry system. Tasks are run in automated sessions where SynapseOS is given the task prompt as a natural language instruction and the system's output is compared against the expected outcome defined in the benchmark.

b) User Study

The user study is conducted in a controlled laboratory setting with two machines: the SynapseOS evaluation machine (Debian 13, conversational interface) and the baseline machine running the participant's native OS — Windows 11, macOS, or Linux with a GNOME desktop — as determined during screening. Each participant completes the full task set on both machines under a within-subjects design, with condition order counterbalanced across participants to mitigate learning and carryover effects. Specifically, half of the participants complete the SynapseOS condition first, and the remaining half complete the native OS condition first. Screen capture software records activity on both machines; the SynapseOS session logger supplements the recording for the SynapseOS condition.

The study involves 20 participants divided into two groups:

- **Novice users (n = 10):** participants with limited computing experience, capable only of basic tasks (web browsing, document editing) on any OS, and no prior command-line familiarity, recruited from the general university population
- **Power users (n = 10):** participants with demonstrated proficiency in computing on their primary OS — capable of performing file management, system monitoring, and process administration tasks efficiently — recruited from the university's information technology and computer science programs

Participants are recruited through university channels. Informed consent is obtained from all participants prior to the session. Ethics approval is secured before recruitment begins. Participant data is anonymized at the point of collection.

After completing each condition, participants fill out the SUS and NASA-TLX questionnaires. After completing both conditions, a brief semi-structured interview is conducted to gather qualitative feedback.

1.3 When to Gather

Data collection proceeds in two phases:

- **Phase 1 — Pilot Study:** A pilot study is conducted with a small group of five participants prior to the main study. The pilot study serves to calibrate the task difficulty, validate the data collection instruments, and identify procedural issues before the main study begins. Instruments are revised based on pilot findings.
- **Phase 2 — Main User Study:** The main user study is conducted following pilot study completion and ethics approval. The specific date range is determined by the ethics review timeline and participant availability.

Benchmark task evaluation runs continuously throughout the development phase of SynapseOS, independently of the user study timeline. This allows for iterative assessment of system performance as the implementation matures.

1.4 Why

The OSWorld benchmark is used as the primary evaluation standard because it is the only existing benchmark that evaluates computer-using agents in real operating system environments with real applications [17]. Its use allows SynapseOS results to be situated within the existing literature.

A custom cross-platform task suite is necessary because OSWorld's task coverage does not represent the human-goal-framed system tasks relevant to this study — file organization, system monitoring, process management, and text processing — achievable both through SynapseOS natural language commands and through participants' native OS tools (Windows, macOS, or Linux graphical desktop). Tasks are scoped to operations achievable via bash on SynapseOS and via native graphical or terminal tools on the comparison machine.

The within-subjects design is chosen because it allows each participant to serve as their own control, producing a direct comparison between conditions while requiring a smaller participant pool than a between-subjects design. Counterbalancing controls for the primary threat to internal validity: the learning effect, wherein participants may perform better in the second condition simply due to familiarity with the tasks.

The expert baseline design — using each participant's primary OS as Condition B rather than a fixed Linux graphical desktop — is chosen because a fixed Linux baseline would confound interface quality with OS familiarity. The overwhelming majority of participants have never used a Linux desktop; performance under that condition would reflect unfamiliarity with a foreign environment, not a meaningful comparison of interface paradigms. The stronger and more ecologically valid test is whether SynapseOS can compete with what participants already know and use daily. A positive result against a participant's native environment is a more compelling argument for SynapseOS than a win against a system participants are encountering for the first time.

The local-SLM-first design is a principled architectural choice, not a fallback. A system that replaces the Linux desktop userland necessarily observes everything the user does at the OS level: every command issued, every file opened, every window on screen. Routing this data to a cloud inference API gives an external provider continuous visibility into the user's computing activity — a privacy exposure that is categorically inappropriate for a system positioned as a userland replacement. On-device inference eliminates this exposure: no user data leaves the machine regardless of what the user is doing.

Cloud model access is preserved as an explicit opt-in. A model-agnostic backend interface accepts user-supplied API keys to substitute the local SLM with a hosted frontier model for users who choose that tradeoff. This design treats privacy as the default and cloud connectivity as a deliberate user choice. It also ensures reproducible evaluation conditions independent of API availability, rate limits, or cost.

The dual-population design — novice and power users — directly addresses the gap identified in Chapter 1, where no prior work has evaluated a conversational interface across user populations with differing technical backgrounds [17]. The three data categories align with the three research questions: RQ1 is addressed by the system design and benchmark task coverage; RQ2 is addressed by system performance telemetry (command execution latency, intent parsing accuracy, undo and confirmation events); and RQ3 is addressed by the user study metrics (task completion time, error rate, satisfaction scores).

1.5 Participant Screening Criteria

Participants are assigned to the novice or power user group based on a pre-study screening questionnaire administered before the session begins. The questionnaire assesses the following dimensions:

- Years of general computer use
- Primary OS — the operating system used as the primary daily computing environment (Windows / macOS / Linux); must average four or more hours of use per day to qualify
- Self-reported proficiency on primary OS (basic / intermediate / advanced)
- Self-reported command-line familiarity (none / basic / intermediate / advanced)
- Prior exposure to conversational AI interfaces (chatbots, voice assistants, LLM tools)

Participants are classified as **novice users** if they report basic proficiency on their primary OS, computing use limited to web browsing and document editing, and no prior command-line experience on any platform. Participants are classified as **power users** if they report advanced proficiency on their primary OS — capable of performing file management, process monitoring, and system administration tasks efficiently using that OS's native tools — regardless of which OS that is. Participants who do not clearly satisfy either classification are excluded. Primary OS and prior exposure to conversational AI interfaces are recorded for all participants as covariates and grouping variables in the analysis.

1.6 Task Design

The custom cross-platform task suite is designed to cover human-level system tasks achievable both through SynapseOS natural language commands and through native tools on the participant's primary OS. Task design follows four principles:

1. **Ecological validity** — tasks reflect actions a real user would perform in their everyday computing environment: finding and organizing files, monitoring system resources, managing processes, and handling text data

2. **Cross-platform equivalence** — all tasks are defined as human-level goals achievable both through SynapseOS natural language commands (mapped to bash) and through native tools on the participant's primary OS (File Explorer, Task Manager, Finder, Activity Monitor, GNOME, or equivalent). The method of achievement differs by platform; the human goal does not
3. **Difficulty stratification** — tasks are rated at three difficulty levels (simple, moderate, complex) based on the number of steps required and the specificity of the target operation
4. **Population appropriateness** — both participant groups receive the same task set, with task wording written in plain language accessible to novice users and free of OS-specific or CLI-specific terminology

The custom task suite comprises tasks organized into four categories: file search and organization, system and process monitoring, application and package management, and text and data processing. All tasks are defined at the human-goal level — achievable via SynapseOS natural language on the evaluation machine and via native graphical or terminal tools on the comparison machine. Each task is documented with a unique identifier, a natural language prompt as presented to the participant, the expected outcome, and the assigned difficulty level. Tasks are validated during the pilot study and revised as necessary before the main study begins.

SECTION 2: PROCEDURE / EXPERIMENT

2.1 System Design and Implementation

The construction of SynapseOS proceeds in three parallel workstreams: the conversational interface layer, the execution pipeline, and the session manager. These components are developed iteratively and integrated into a unified Linux session that replaces the conventional graphical desktop.

a) Conversational Interface Layer

The conversational surface is implemented as a persistent, full-screen chat interface that serves as the primary and sole user-facing interface when SynapseOS is active as the session manager. The interface accepts natural language input, displays system responses and command output, and maintains conversational context across the session. An intent parsing pipeline translates each user utterance into a shell command for execution.

SynapseOS is designed around a locally-hosted small language model (SLM) as the default inference backend. Because the system operates at the OS level — processing every natural language command, observing terminal output, file paths, command history, and the user's interaction patterns — routing inference through a cloud API would give an external provider continuous visibility into the user's computing activity. SynapseOS therefore defaults to on-device inference: all language processing runs locally, with no user data transmitted externally.

The inference backend is model-agnostic by design. Users may optionally supply an API key for a supported cloud provider to substitute the local SLM with a hosted frontier model; this is entirely at the user's discretion. The default configuration requires no API key, no internet connection, and no per-query cost, making SynapseOS fully operational in offline and air-gapped environments.

The primary SLM is Qwen2.5-Coder-3B-Instruct, an open-weights code-specialized model that demonstrated the strongest natural language-to-bash translation accuracy among models below 7B parameters in controlled evaluation [2]. At 4-bit quantization, the model requires approximately 1.8 GB of memory and runs on the target Debian 13 (Trixie) hardware without a dedicated GPU. The model is fine-tuned via Low-Rank Adaptation (LoRA) on the NL2Bash corpus and a custom SynapseOS task suite to maximize intent parsing accuracy.

Intent parsing accuracy is evaluated following the methodology established by Westenfelder et al. [2], who assessed NL2Bash performance through execution-based scoring rather than syntax-level string matching: each generated command is executed in a controlled environment and its output compared against the expected result, with ambiguous cases resolved through LLM-assisted functional equivalence assessment at 95% confidence. Westenfelder et al. further identified that more than half of the InterCode NL2Bash dataset contains erroneous entries, and constructed larger, manually verified datasets for evaluation. Applying their methodology across three conditions — zero-shot baseline, prompt engineering, and LoRA fine-tuning — they found that fine-tuning on small models produced accuracy gains sufficient to approach the zero-shot baseline performance of significantly larger models. Qwen2.5-Coder-3B-Instruct achieved 58% NL2Bash accuracy with prompt engineering alone — the highest among models below 7B parameters in their evaluation [2] — motivating both its selection as the SynapseOS intent parsing model and the adoption of their execution-based evaluation pipeline for measuring accuracy in this study.

b) Execution Pipeline

Once the intent parser produces a shell command, SynapseOS passes it through two safety mechanisms before execution. First, the command is classified by reversibility: operations that are irreversible — such as file deletion, system configuration changes, or destructive package operations — trigger a confirmation prompt requiring explicit user approval before execution proceeds. Reversible operations are logged to the undo stack, enabling the user to request rollback at any point through the conversational interface. Second, the approved command is dispatched to a bash subprocess. Standard output and standard error are captured and rendered in the conversational surface as the system's response to the original natural language input.

SynapseOS executes only shell-expressible operations. The system's capability boundary is the Linux command-line toolchain: file management, process control, text processing, package management, network configuration, and any application that exposes a command-line interface. GUI-only interactions — clicking buttons in graphical applications, navigating web page elements, or interacting with visual-only interfaces — are explicitly outside scope. This boundary is intentional: it constrains the research to a tractable, verifiable claim that the system can be built, evaluated, and defended within the thesis timeline.

c) Session Manager

SynapseOS is implemented as a Linux session manager that launches in place of the conventional desktop environment. The session manager maintains a persistent conversational surface, a structured undo log that records reversible operations for potential rollback, and a confirmation policy that intercepts irreversible operations — such as file deletion or system configuration changes — and prompts the user for explicit approval before execution.

2.2 Data Pre-processing

Prior to analysis, all collected data undergoes the following processing steps:

- **Task normalization:** task descriptions and expected outcomes are standardized across both the OSWorld benchmark and the custom task suite to ensure consistent evaluation criteria
- **Log parsing and event extraction:** raw session logs from SynapseOS telemetry are parsed to extract per-task metrics — completion time, error events, and undo events

- **Metric computation:** task completion time is computed as the elapsed time between task prompt presentation and task completion event; error rate is computed as the ratio of failed or incorrect task outcomes to total task attempts per participant per condition; SUS and NASA-TLX scores are computed according to their respective standard scoring formulas
- **Participant data anonymization:** all participant records are anonymized at the point of extraction, replacing identifying information with participant ID codes prior to any analysis

2.3 Validation

The following validation procedures are applied to ensure the reliability and accuracy of the system and the study instruments:

- **Pilot study:** conducted with five participants prior to the main study to validate the task set difficulty, the questionnaire instruments, and the session procedure; findings from the pilot study are used to revise instruments and procedures before the main data collection begins
- **Intent parsing accuracy:** evaluated by comparing the system's parsed intent representation against a human-annotated ground truth for each task utterance; accuracy is computed as the proportion of utterances where the parsed intent matches the annotated intent on the first attempt
- **Confirmation policy effectiveness:** evaluated by verifying that all irreversible operations in the task set trigger the confirmation prompt as designed, and that no irreversible operations execute without user approval
- **Undo success rate:** evaluated by executing a representative set of reversible operations and verifying that the undo log correctly restores the prior system state in each case

2.4 Study Session Procedure

Each participant session follows a standardized procedure conducted in the following order:

1. **Briefing:** The session facilitator explains the study purpose, session structure, and participant rights. The participant reads and signs the informed consent form. The pre-study screening questionnaire is administered to confirm group classification.

2. **System familiarization:** For the SynapseOS condition, the participant is given a five-minute orientation to the conversational interface and shown how to enter natural language commands. For the native OS condition, no familiarization is provided — participants use their home environment as they normally would. No task-specific training is provided under either condition.

3. **Task completion:** The participant completes the full task set under the assigned condition. Tasks are presented one at a time via a printed task prompt sheet. The session logger captures start and end times automatically. Screen recording captures all on-screen activity. The participant is instructed to think aloud throughout.

4. **Post-condition questionnaire:** Immediately after completing all tasks under a condition, the participant completes the SUS and NASA-TLX questionnaires for that condition.

5. **Condition changeover:** If the participant has not yet completed both conditions, a short break is offered, the participant moves to the other machine (SynapseOS or native OS), and the session returns to step 2.

6. **Post-study interview:** After both conditions are completed, the facilitator conducts a semi-structured interview of approximately 10–15 minutes covering perceived ease of use, trust in the conversational interface, preference between conditions, and open-ended feedback.

7. **Debrief:** The facilitator discloses the full study objectives, answers participant questions, and thanks the participant. Session data is anonymized immediately upon session close.

Total estimated session duration per participant: 90–120 minutes.

2.5 Ethical Considerations

This study involves human participants and is subject to ethics review and approval by the Institutional Review Board (IRB) or equivalent ethics committee of Mapúa University – Makati prior to the commencement of participant recruitment. The following ethical principles are observed throughout the study:

- **Informed consent:** All participants receive a written information sheet describing the study purpose, procedures, data collected, and their rights. Participation is entirely voluntary; participants may withdraw at any time without penalty or consequence.

- **Anonymization:** Participant identities are not recorded. Each participant is assigned an anonymous ID code at the point of enrollment. All collected data — session logs, questionnaire responses, and interview transcripts — are stored and analyzed using participant ID codes only.
- **Data minimization:** Only the data types specified in Section 1.1 are collected. No identifying metadata — names, email addresses, or device identifiers — is retained beyond the consent confirmation step.
- **Data security:** All collected data is stored on password-protected institutional infrastructure accessible only to the research team. Interview recordings are deleted after transcription and verification.
- **Confidentiality:** Study findings are reported in aggregate. No individual participant's data is disclosed in any form that could enable identification.
- **Right to withdraw:** Participants may discontinue the session and request withdrawal of their data at any point during or after the session, up to the point at which anonymization makes individual data unidentifiable.

SECTION 3: SYSTEM TESTING

3.1 Evaluation Dataset

The system is evaluated against two datasets. The first is the OSWorld benchmark task suite [17], which provides a standardized set of real-computer tasks for direct comparison with results reported in the existing literature. The second is the custom cross-platform task suite developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing — each task defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine.

The user study evaluation involves 20 participants: 10 novice users and 10 power users. The baseline condition is each participant's primary operating system — Windows 11, macOS, or Linux with GNOME desktop — operated on a dedicated baseline machine under the same laboratory conditions and the same human-goal task set.

3.2 Evaluation Profile

The study employs a within-subjects design with two conditions:

- **Condition A:** SynapseOS (conversational interface layer on the Debian 13 evaluation machine)
- **Condition B:** Participant's primary operating system (Windows 11, macOS, or Linux with GNOME desktop, as determined by pre-study screening, on the dedicated baseline machine)

All 20 participants complete both conditions. Condition order is counterbalanced: 10 participants complete Condition A first, and 10 participants complete Condition B first. This counterbalancing controls for order effects and task familiarity.

The two participant groups — novice users and power users — are analyzed both in aggregate and separately to determine whether the effect of the interface condition differs across experience levels. Additionally, primary OS (Windows, macOS, Linux) is recorded as a secondary grouping variable to determine whether results vary across participant backgrounds. The primary analysis is the pooled within-subjects comparison across all 20 participants; per-OS subgroup analyses are treated as exploratory given the smaller per-group sample sizes expected at $n = 20$.

3.3 Criteria and Formula

The following metrics and formulas are used to evaluate SynapseOS against the baseline:

a) Task Completion Time

Mean task completion time is computed per condition per participant group. The primary statistical test is the paired-samples t-test for normally distributed data or the Wilcoxon signed-rank test for non-normally distributed data, applied at a significance level of $\alpha = 0.05$. Effect size is reported using Cohen's *d*.

b) Error Rate

Error rate is computed as the proportion of task attempts resulting in failure or incorrect outcome:

$$\textit{Error Rate} = (\textit{Failed Tasks} + \textit{Incorrect Outcomes}) / \textit{Total Task Attempts}$$

Error events are further categorized by type: intent parsing errors (incorrect bash command generated), execution errors (bash subprocess failure), and recovery failures (unsuccessful undo or retry). The same paired statistical test is applied to error rate differences between conditions.

c) User Satisfaction

SUS scores are computed using the standard SUS scoring formula: for each of the ten SUS items, the score contribution is computed (odd-numbered items: score minus 1; even-numbered items: 5 minus score), the contributions are summed, and the total is multiplied by 2.5 to produce a score on a 0–100 scale. A SUS score above 68 is considered above-average usability.

NASA-TLX scores are computed by averaging the six subscale ratings (Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, Frustration) to produce a composite workload score per participant per condition.

Qualitative interview data is analyzed using thematic analysis to identify recurring themes in participant perceptions of both conditions.

d) Statistical Significance and Effect Size

All primary comparisons between conditions are tested at $\alpha = 0.05$. Effect size is reported using Cohen's *d*, with thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Minimum sample size justification is based on an a priori power analysis targeting 80% statistical power at the specified significance level and a medium effect size ($d = 0.5$), which yields a minimum of 17 participants for a within-subjects design — satisfied by the 20-participant sample.

3.4 Data Analysis Plan

Quantitative analysis proceeds in three stages. First, descriptive statistics — means, standard deviations, and 95% confidence intervals — are computed for all primary metrics (task completion time, error rate, SUS score, NASA-TLX score) per condition and per participant group. Second, the Shapiro-Wilk test is applied to assess normality of each distribution before selecting the appropriate inferential test: the paired-samples t-test for normally distributed data, or the Wilcoxon signed-rank test for non-normally distributed data. Third, effect size (Cohen's *d*) is computed for all statistically significant comparisons.

Subgroup analysis is conducted separately for the novice and power user groups to determine whether the effect of the interface condition differs across populations. The interaction between interface condition and user group is examined to assess whether SynapseOS produces differential effects depending on technical background.

Qualitative data from post-study interviews is analyzed using reflexive thematic analysis following the six-phase procedure described by Braun and Clarke [24]: familiarization with the data, initial code generation, theme search, theme review, theme definition, and write-up. Two members of the research team independently code a random subset of transcripts; inter-rater agreement is assessed and disagreements are resolved through discussion before finalizing the codebook. Themes are reported alongside representative participant quotations.

3.5 Threats to Validity

The following threats to validity are identified and the mitigations applied in this study are described.

Internal validity:

- **Learning effect:** Participants may perform better on the second condition simply due to task familiarity, independent of interface quality. *Mitigation:* condition order is counterbalanced across participants so that learning effects are distributed equally across both conditions and do not systematically favor either.
- **Demand characteristics:** Participants may behave differently from natural use because they know they are being observed. *Mitigation:* participants are instructed to complete tasks as they naturally would; the specific hypotheses of the study are withheld until the post-study debrief to reduce social desirability bias.
- **Fatigue:** Completing the full task set twice in a single session may degrade performance in the second condition independent of interface quality. *Mitigation:* a short break is offered between conditions and total session length is bounded to approximately 120 minutes.

External validity:

- **Sample generalizability:** The study uses a university-recruited sample of 20 participants across three primary OS backgrounds (Windows, macOS, Linux), which limits direct generalization to the broader population of computer users. Findings should be interpreted as indicative rather than definitive; replication with larger and more diverse samples is recommended as a direction for future work.
- **Task generalizability:** The evaluation is bounded by OSWorld coverage and the custom task suite developed for this study. Computing workflows outside this coverage are not evaluated, and results may not extend to all possible use cases.

- **Platform specificity:** SynapseOS is evaluated on a single hardware and software configuration running Debian 13 (Trixie). Performance characteristics on different hardware, display configurations, or Linux distributions may differ from the reported results.

REFERENCES

[2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. https://arxiv.org/abs/2502.06858

[3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. https://arxiv.org/abs/2506.13028

[5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. https://doi.org/10.1145/3654777.3676401

[16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO^2: The Desktop AgentOS. *arXiv:2504.14603*. https://arxiv.org/abs/2504.14603

[17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2404.07972

[22] Han, W., Zeng, Z., Huang, J., Jiang, S., et al. (2025). GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. *arXiv:2506.11127*. https://arxiv.org/abs/2506.11127

[23] Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., & Ghadar, S. (2025). R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 9669–9685. https://doi.org/10.18653/v1/2025.findings-acl.501

[24] Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, *3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa